Guia de uso de la calculadora.

El cliente funciona recibiendo como argumento la operación a realizar de la forma ./calc_cliente *host operación*

Operación es un único argumento que va entrecomillado y en su interior se deben encontrar los datos y/u operadores separados cada uno por un espacio. Los espacios son necesarios para poder tokenizar la entrada.

Ej: ./calc_cliente localhost “2 + 3”

La calculadora permite realizar varias operaciones de forma simultanea, siguiendo el orden de operaciones PEMDAS. Para ello simplemente añade las operaciones dentro de las comillas dobles.

Ej: ./calc_cliente localhost “2 + 3 * (5 – 2)”

```
david@PORTATIL-DAVID:/mnt/c/Users/Usuario/OneDrive/Escritorio/DSD_P1$ ./calc_server
Resultado: 3.4
```

```
david@PORTATIL-DAVID:/mnt/c/Users/Usuario/OneDrive/Escritorio/DSD_P1$ ./calc_client localhost "3 + 2 / 5"
Resultado: 3.4
```

De forma interna, el orden de las operaciones se calcula siguiendo el algoritmo shunting yard de Dijkstra para pasar la operación en forma infija a postfija, con lo cual se establece la prioridad de operaciones teniendo en cuenta paréntesis, multiplicaciones / divisiones y sumas y restas.

```c
static int precedencia(char op)
{
    return (op == '*' || op == '/') ? 2 : 1;
}

static int shunting_yard(char *toks[], int n_toks, token output[])
{
    char op_stack[n_toks];
    int  op_top = 0;
    int  out_n  = 0;

    for (int i = 0; i < n_toks; i++) {
        char *t = toks[i];

        if (t[0] == '(') {
            op_stack[op_top++] = '(';

        } else if (t[0] == ')') {
            while (op_top > 0 && op_stack[op_top-1] != '(') {
                output[out_n].type       = TOKEN_OP;
                output[out_n].token_u.op = (int)op_stack[--op_top];
                out_n++;
            }
            if (op_top == 0) {
                fprintf(stderr, "Error: parentesis no balanceados\n");
                return -1;
            }
            op_top--;   /* descartar '(' */
```

```c
        } else if (t[0]=='+' || t[0]=='-' || t[0]=='*' || t[0]=='/') {
            while (op_top > 0 &&
                   op_stack[op_top-1] ≠ '(' &&
                   precedencia(op_stack[op_top-1]) ≥ precedencia(t[0])) {
                output[out_n].type        = TOKEN_OP;
                output[out_n].token_u.op = (int)op_stack[--op_top];
                out_n++;
            }
            op_stack[op_top++] = t[0];

        } else {
            output[out_n].type        = TOKEN_NUM;
            output[out_n].token_u.num = atof(t);
            out_n++;
        }
    }

    while (op_top > 0) {
        if (op_stack[op_top-1] == '(') {
            fprintf(stderr, "Error: parentesis no balanceados\n");
            return -1;
        }
        output[out_n].type        = TOKEN_OP;
        output[out_n].token_u.op = (int)op_stack[--op_top];
        out_n++;
    }

    return out_n;
}
```

Una vez se tienen los tokens separados en notación postfija se le pasa al servidor, que simplemente va metiendo en una pila los datos según le llegan y saca los dos datos más recientes cuando se encuentra un operador. El resultado final es el único dato que queda en la pila al terminar de procesar todos los datos de entrada.

```c
    for (int i = 0; i < expr.expr_len; i++) {
        token *t = &expr.expr_val[i];

        if (t->type == TOKEN_NUM) {
            stack[top++] = t->token_u.num;
        } else {
            b = stack[--top];
            a = stack[--top];

            switch ((char)t->token_u.op) {
                case '+':
                    res = a + b;
                    break;

                case '-':
                    res = a - b;
                    break;

                case '*':
                    res = a * b;
                    break;

                case '/':
                    if (b == 0.0) {
                        result.errnum = 10;
                        return &result;
                    }
                    res = a / b;
                    break;
            }
            stack[top++] = res;
        }
    }

    result.calc_res_u.res = stack[0];
```

Como se puede ver, el caso de dividir por 0 se gestiona en el servidor, mientras se procesa la entrada.

El tipo de dato que se le da como parámetro a la función es una “expresión” definida por ser un array de tokens. Un token puede ser del tipo numero o del tipo operador.

```
const MAX_TOKENS = 64;

enum token_type { TOKEN_NUM = 0, TOKEN_OP = 1 };

union token switch (token_type type) {
    case TOKEN_NUM: double num;
    case TOKEN_OP:  int    op;
};

typedef token expr<MAX_TOKENS>;
```